

# Rapport de Développement Web : Application Televerie

## Projet réalisé par :

Facundo ARITO, Juan BAUDINO, Juan DE MIGUEL, Julian RAYES

Enseignant : Sébastien KUBICKI

École Nationale d'Ingénieurs de Brest (ENIB)

## Introduction Générale

La société prestataire en charge des équipements de laverie des résidences CROUS de Brest nous a contactés pour résoudre un problème d'organisation majeur dans leurs locaux. Ce document s'inscrit dans la continuité de notre précédent rapport axé sur la conception centrée utilisateur (création des personas, idéation, wireframes, maquettes et prototype).

L'objectif de ce nouveau rapport est de présenter la programmation technique du Frontend de l'application "Televerie". Nous y détaillerons la traduction de nos maquettes en code fonctionnel, en utilisant les langages web standards : HTML, CSS et JavaScript.

## Infrastructure et Organisation du Projet

La mise en place du projet a commencé par la création d'un dépôt structuré de manière modulaire. L'objectif est de séparer clairement les différentes responsabilités de l'application pour faciliter le développement. Voici l'arborescence actuelle du projet avec ses composants principaux :

```
.
|-- icons/
|-- img/
|   '-- profile_pics/
|-- index.html
|-- js/ (Logique de fonctionnement et scripts)
|   |-- app.js (Système de routage global)
|   |-- services/
|   '-- views/ (Initialisation des écrans)
|-- rapport/ (Documentation technique)
|-- README.md
|-- ressources/
|   '-- data/ (Base de données locale JSON)
|-- src/ (Fragments HTML et composants)
|   |-- bottom-nav.html
|   '-- views/ (Fichiers HTML des vues)
'-- styles/ (Feuilles de style CSS)
    |-- bottom-nav.css
    |-- global.css
    '-- views/
```

# Comment lancer l'application

Pour tester l'application "Televerie" sur votre ordinateur, plusieurs méthodes sont possibles. Veuillez suivre ces étapes selon la configuration de votre machine :

## Méthode 1 : Ouverture directe

La méthode la plus simple consiste à faire un double-clic sur le fichier `index.html`. Cela ouvrira l'application directement dans votre navigateur par défaut.

## Méthode 2 : Configuration sur Firefox

Si la première méthode ne fonctionne pas (parfois à cause des sécurités du navigateur bloquant les scripts locaux), nous recommandons d'utiliser Firefox :

- Ouvrez Firefox et tapez `about:config` dans la barre d'adresse.
- Acceptez l'avertissement de sécurité.
- Cherchez le paramètre `privacy.file_unique_origin`.
- Changez sa valeur de `true` à `false`.
- Rechargez la page `index.html`.

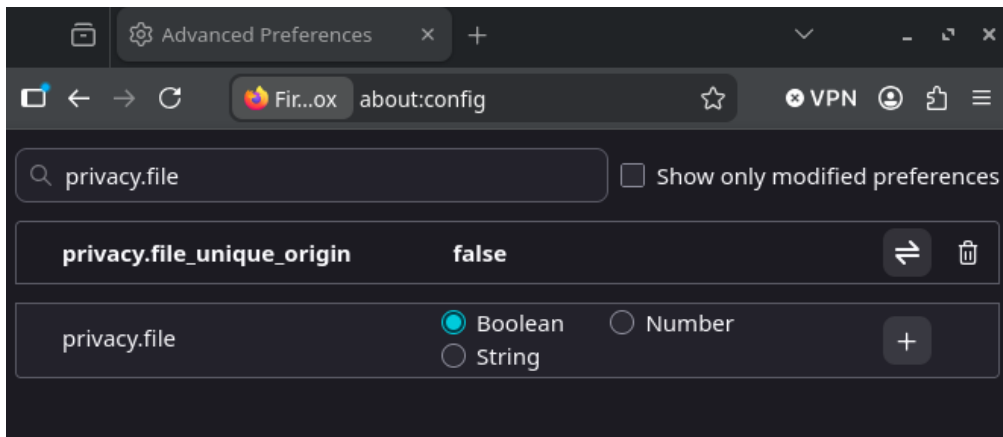


FIGURE 1 – Configuration du paramètre de confidentialité sur Firefox.

## Méthode 3 : Lancer un serveur local (Python)

Si les méthodes précédentes échouent, vous pouvez simuler un environnement de serveur web local via le terminal :

- Ouvrez un terminal (ou l'invite de commandes).
- Naviguez jusqu'au dossier racine qui héberge le fichier `index.html`.
- Lancez la commande suivante : `python -m http.server`
- Ouvrez votre navigateur et accédez à l'adresse : `localhost:8000`

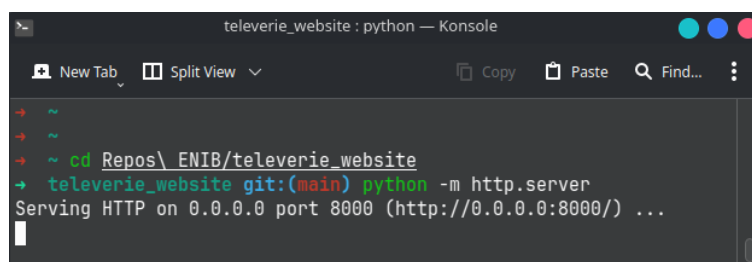


FIGURE 2 – Lancement du serveur local avec Python.

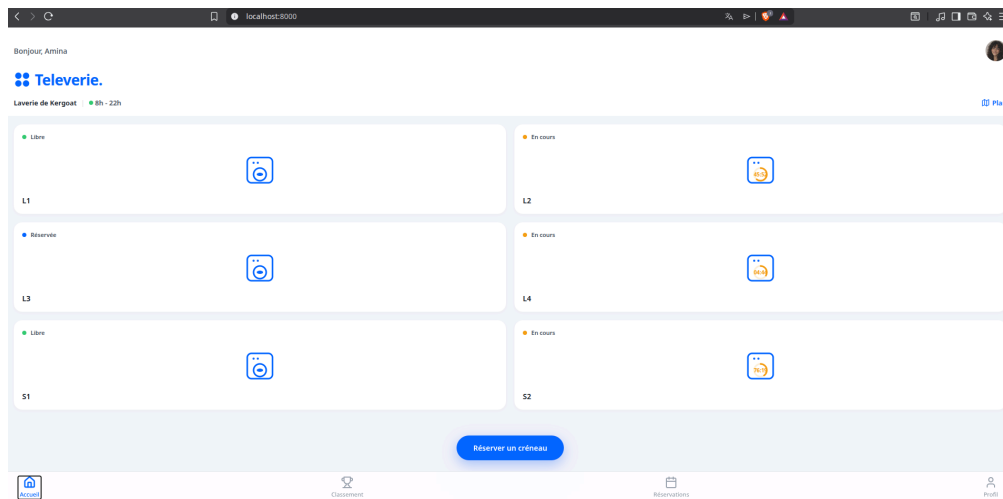


FIGURE 3 – Accès au serveur local.

## Affichage en mode Mobile-First

Une fois sur la page, l'application s'affichera à l'écran comme illustré avant, en mode PC. Cependant, étant donné que "Televerie" est conçue avec une approche "Mobile-First", vous devez activer la vue mobile pour profiter d'une expérience optimale et fidèle au design :

- Appuyez sur **Ctrl + Shift + I** (ou la touche **F12**) pour ouvrir les outils de développement du navigateur.
- Activez l'affichage "Appareil mobile" (souvent représenté par une icône de téléphone/tablette).
- Dans le menu déroulant en haut de l'écran, sélectionnez la vue **iPhone 14** ou **iPhone 14 Pro Max**.

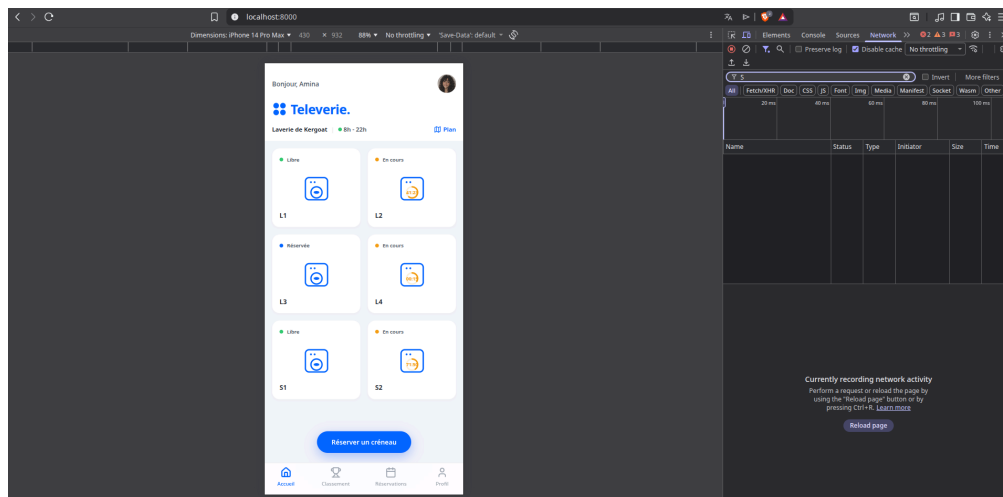


FIGURE 4 – Sélection de la vue iPhone 14 dans les outils de développement.

# Page d'Accueil

## Fonctionnalités de l'Accueil

La page d'accueil est le tableau de bord principal de l'application. Son but est de donner à l'étudiant toutes les informations les plus importantes en un seul coup d'œil, pour qu'il ne perde pas de temps.

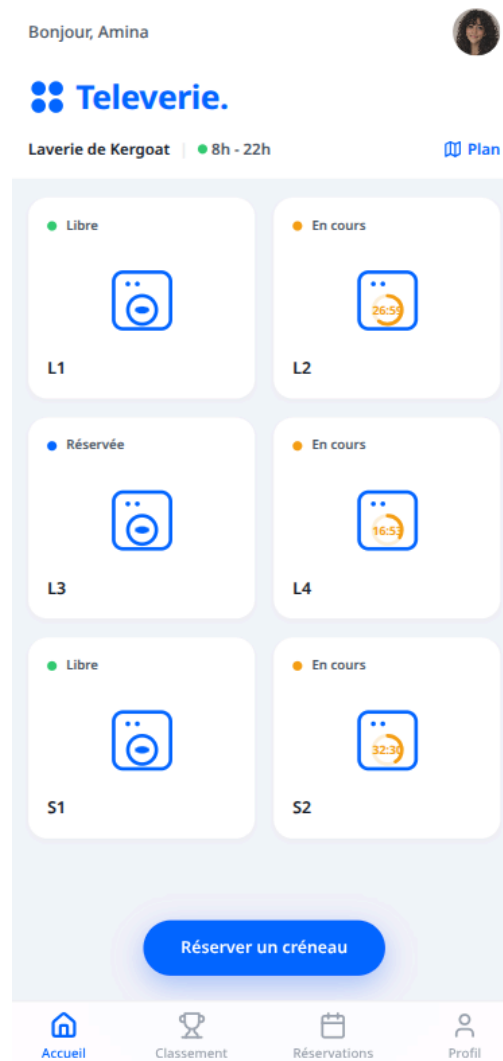


FIGURE 5 – Aperçu de la page d'accueil.

## L'en-tête personnalisé

En haut de l'écran, l'utilisateur est accueilli par son prénom (par exemple, "Bonjour, Amina") et sa photo de profil. Juste en dessous, l'application indique le nom de la laverie actuellement sélectionnée (comme "Laverie de Kergoat") et ses horaires d'ouverture (8h - 22h). Cet espace sert à rassurer l'utilisateur sur le lieu qu'il est en train de consulter. De plus, un bouton "Plan" est placé juste à côté pour permettre d'ouvrir rapidement une carte des résidences.

## L'état des machines en direct

Le centre de la page est composé d'une grille montrant 4 machines à laver et 2 sèche-linges. Pour rendre l'interface très visuelle et facile à comprendre, nous avons utilisé un code

couleur simple pour indiquer le statut de chaque équipement :

- **Vert (Libre)** : La machine est prête à être utilisée immédiatement.
- **Orange (En cours)** : La machine est occupée. Pour aider l'étudiant à calculer son temps, une animation circulaire avec un compte à rebours indique exactement les minutes restantes avant la fin du programme.
- **Bleu (Réservée)** : La machine est bloquée et attend l'arrivée de l'étudiant qui a réservé ce créneau.

## L'action rapide (Bouton principal)

Enfin, en bas de l'écran, un grand bouton bleu "Réserver un créneau" flotte au-dessus du contenu. Il s'agit de l'action principale de l'application. Un simple clic sur ce bouton connecte directement l'utilisateur au calendrier de réservation, ce qui rend l'expérience beaucoup plus fluide et rapide.

## Navigation et Fenêtres Contextuelles (Pop-ups)

Afin de rendre l'application plus fluide et d'éviter de recharger la page entière, nous avons intégré des "pop-ups" (ou fenêtres modales) directement sur la page d'accueil. Cette technique améliore considérablement l'expérience utilisateur (UX) car elle donne l'impression d'utiliser une véritable application mobile native. L'utilisateur reste toujours dans son contexte sans perdre le fil de sa navigation.

### Pop-up de la Carte (Plan)

En cliquant sur le bouton "Plan" situé en haut à droite de l'écran, une fenêtre superposée s'ouvre avec une carte interactive Google Maps intégrée. Cette carte montre avec précision 3 points (pins) rouges qui correspondent aux 3 résidences CROUS situées dans Brest Métropole. Cela est très utile pour les étudiants qui souhaitent vérifier la distance ou l'emplacement d'une autre laverie si la leur est complète. La fenêtre se ferme facilement en touchant la croix ou simplement en cliquant sur le fond sombre.

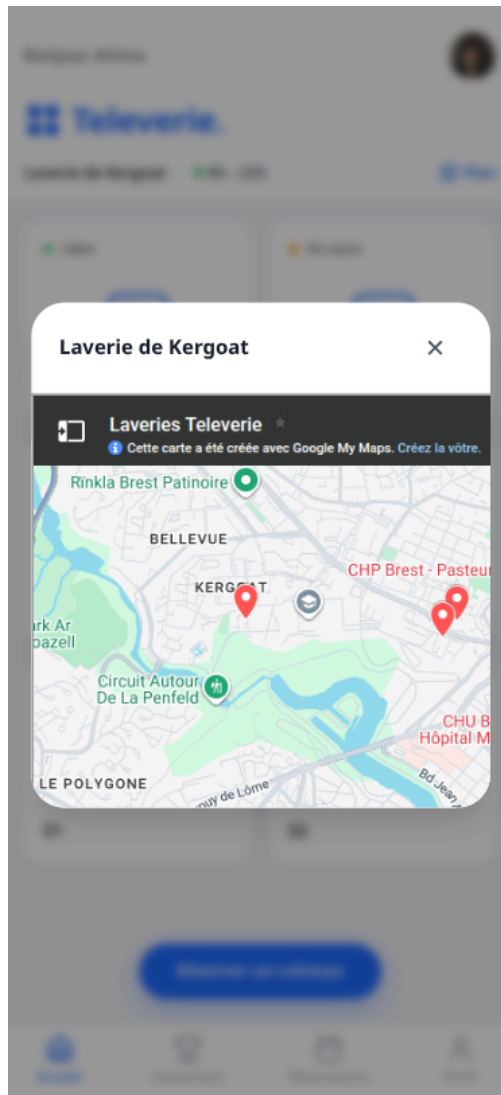


FIGURE 6 – Aperçu de la fenêtre modale affichant la carte interactive des résidences CROUS.

## Pop-up du Profil Rapide

Pour offrir un raccourci pratique, un clic sur la photo de profil ouvre un petit résumé des données de l'utilisateur. Par exemple, pour l'étudiante Amina Rahmani, le pop-up affiche son nom complet, son e-mail universitaire et son numéro de téléphone dans un design épuré avec des cadres blancs arrondis. En bas de ce pop-up, le bouton "Voir mon profil" permet de naviguer instantanément vers la section complète du compte.

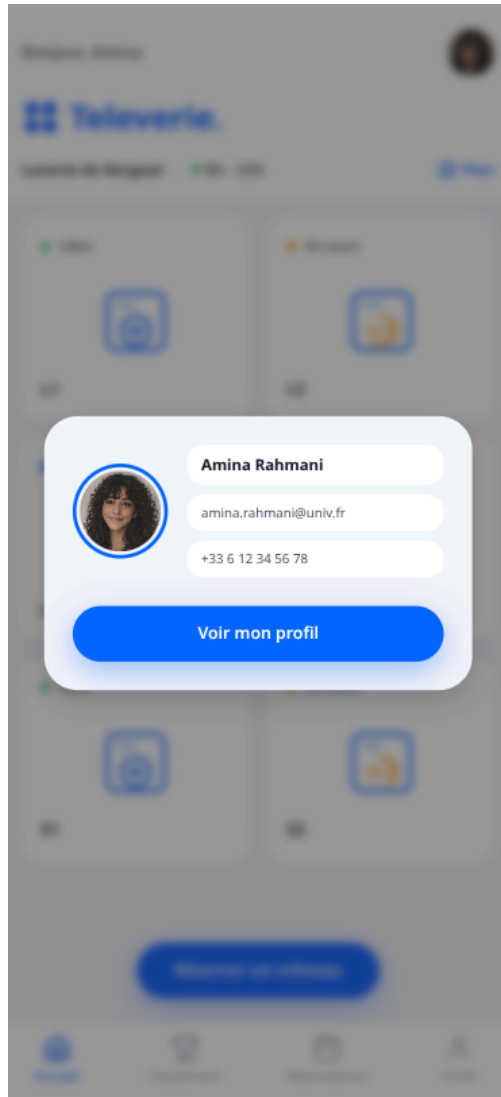


FIGURE 7 – Aperçu du pop-up de profil rapide avec les coordonnées basiques de l'utilisateur.

## Pop-up de Réservation

L'objectif principal de l'application est de permettre à l'utilisateur de réserver un créneau pour une machine spécifique à l'avance. Pour faciliter cette action fondamentale, le grand bouton bleu "Réserver un créneau", situé en bas de l'écran d'accueil, fait le lien direct. Lorsqu'on le touche, il simule une navigation vers l'onglet des réservations et indique au système d'ouvrir automatiquement la fenêtre modale du calendrier. L'étudiant peut ainsi choisir sa date et son heure immédiatement, ce qui lui fait gagner des clics.

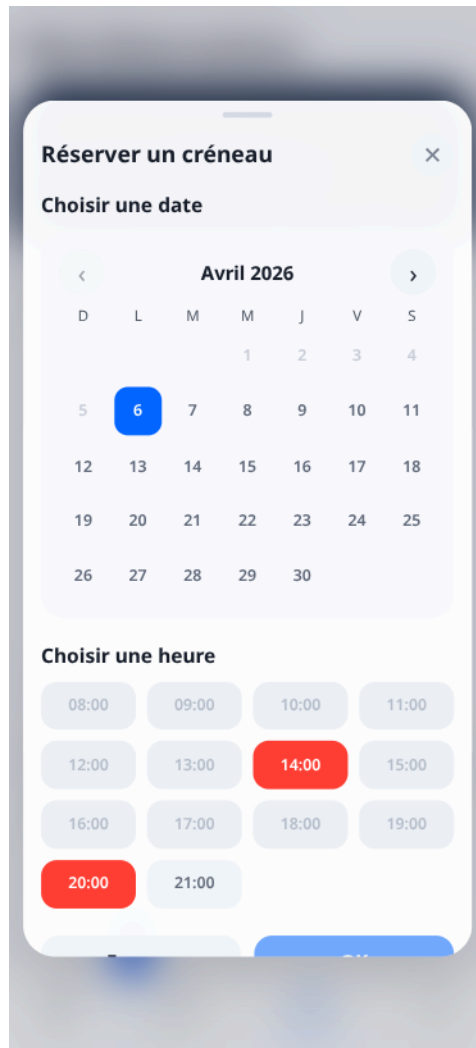


FIGURE 8 – Aperçu de la fenêtre modale de réservation avec le calendrier et les horaires.

## Détails Techniques : Le code `home.js`

La logique interactive de cette page se trouve dans le fichier `home.js`.

### Persistance des données (Le système de mémoire)

L'un des défis techniques était de garder les chronomètres actifs même si l'utilisateur change d'onglet (par exemple, s'il va voir son profil puis revient à l'accueil). Pour résoudre cela, nous utilisons un objet `Map()` appelé `homeMachineEndTimes`. Lorsqu'une machine commence à tourner, le système enregistre son heure de fin dans cette "mémoire". Ainsi, quand la vue `initHomeView()` est rechargée, le code vérifie d'abord si la machine a déjà une heure de fin enregistrée avant de créer un nouveau chronomètre.

### Un calcul du temps générique et automatique

La fonction `updateTimers()` est responsable de faire reculer le temps. Ce qui est intéressant dans notre code, c'est sa flexibilité. La fonction ne cherche pas "le lave-linge 2" ou "le sèche-linge 1". Elle cherche simplement n'importe quel élément HTML qui possède la classe CSS `.en-cours`.



Listing 1 – Extrait de la fonction de mise à jour du temps

```

1 function updateTimers() {
2     // Le code trouve automatiquement les lave-linges ET les seche-
      linges occupes
3     const machines = document.querySelectorAll('.machine-card.en-
      cours');
4
5     machines.forEach(machine => {
6         // Lecture de l'heure de fin directement depuis le HTML
7         const endTimeAttr = machine.dataset.endTime;
8         const remaining = new Date(endTimeAttr).getTime() - Date.now
          ();
9
10        if (remaining <= 0) {
11            // Transformation automatique de l'interface en statut "
              Libre"
12            machine.classList.remove('en-cours');
13            // ... (mise a jour des couleurs SVG)
14        }
15    });
16 }

```

Grâce à cette logique et à l'attribut `data-end-time`, nous pouvons ajouter autant de machines ou de sèche-linges que nous voulons dans le fichier HTML ; le JavaScript les détectera et les animera automatiquement sans avoir besoin de modifier le code.

## Gestion des Pop-ups et Navigation Simulée

Pour l'affichage des fenêtres modales (la Carte et le Profil), nous avons opté pour une manipulation simple du DOM (Document Object Model). Les fenêtres sont déjà présentes dans le HTML mais sont cachées grâce à l'attribut `hidden`. Un clic sur le bouton "Plan" ou sur la photo de profil bascule cet attribut pour les afficher ou les cacher.

Enfin, pour la navigation, nous avons utilisé une technique de "clic simulé" pour respecter l'architecture "Single Page Application" (SPA) de notre projet.

Listing 2 – Exemple de navigation simulée vers les réservations

```

1 const reserveBtn = document.querySelector('.btn-primary-pill');
2 if (reserveBtn) {
3     reserveBtn.addEventListener('click', () => {
4         // 1. On active un signal pour ouvrir le calendrier
              automatiquement
5         window.openBookingModalFromHome = true;
6
7         // 2. On simule un clic sur l'icone "Reservations" du menu en
              bas
8         const navItems = document.querySelectorAll('.bottom-nav__item
              ');
9         if (navItems.length > 2) navItems[2].click();
10    });
11 }

```

Cette méthode permet de réutiliser le code de navigation global (situé dans `app.js`) sans créer de doublons ni de rechargement de page.

## Page de Classement (Ranking)

L'une des sections principales du menu de navigation est l'onglet « Classement ». Cette vue permet de visualiser le score de crédit actuel de l'utilisateur ainsi que sa position relative par rapport aux autres résidents. L'idée fondamentale de notre projet est de gérer la priorité des réservations des machines en fonction de ce score. Ainsi, en cas de requêtes simultanées pour un même créneau ou une même machine, l'application accordera la priorité à l'utilisateur possédant le meilleur score de crédit.

### Score Personnel

L'interface présente en premier lieu la section dédiée au score personnel. Nous y avons intégré une animation circulaire qui se remplit progressivement pour afficher le score cumulé de l'utilisateur de manière dynamique. De plus, selon le nombre de points obtenus, un message contextuel indique la position de l'utilisateur dans le classement global (par exemple, l'appartenance au top 10% des utilisateurs de la résidence).



FIGURE 9 – Affichage dynamique du score personnel de l'utilisateur.

### Classement Global

Dans un second temps, la vue intègre le classement global des utilisateurs (Leaderboard). Afin de séparer la logique de programmation de l'interface web de la base de données des utilisateurs, les informations sont gérées et récupérées via un fichier `.json`. Ce fichier de données stocke les attributs essentiels de chaque résident, tels que le nom, le score, la position dans le classement et le chemin d'accès vers la photo de profil.

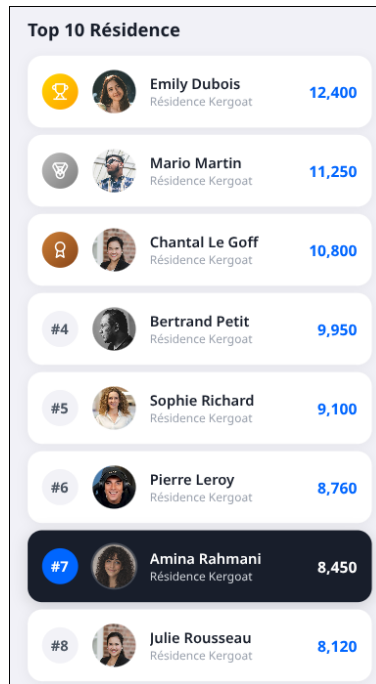


FIGURE 10 – Classement global des utilisateurs alimenté par un fichier JSON.

## Politiques d'Utilisation et Fenêtre Modale d'Information

Enfin, comme illustré sur la vue du score personnel, un bouton d'aide en forme de point d'interrogation est mis à la disposition de l'utilisateur. Un clic sur ce bouton déclenche l'apparition d'une fenêtre contextuelle (pop-up modale) centrée sur l'écran, qui détaille les politiques de l'application. Elle explique concrètement les actions permettant à un utilisateur de gagner ou de perdre des points. Cette approche garantit que l'utilisateur est pleinement informé des comportements attendus dans la laverie.

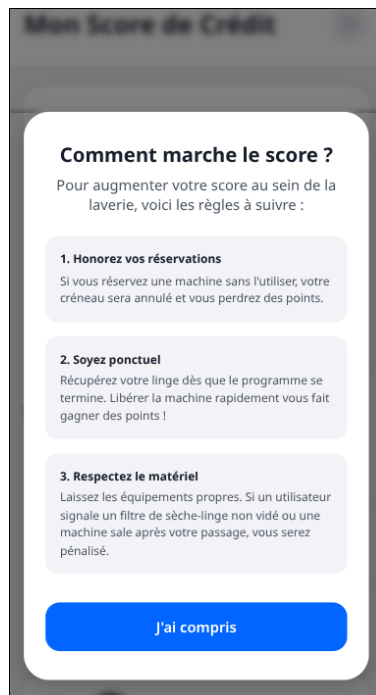


FIGURE 11 – Fenêtre modale détaillant les politiques d'attribution des points.

# Page des Réservations

La page **Réservations** constitue l'espace central de planification de l'application. Alors que la page d'accueil permet surtout de consulter rapidement l'état global de la laverie, cette vue donne à l'étudiant les outils nécessaires pour **anticiper sa lessive, choisir un créneau pertinent et retrouver l'historique détaillé de ses sessions précédentes**.

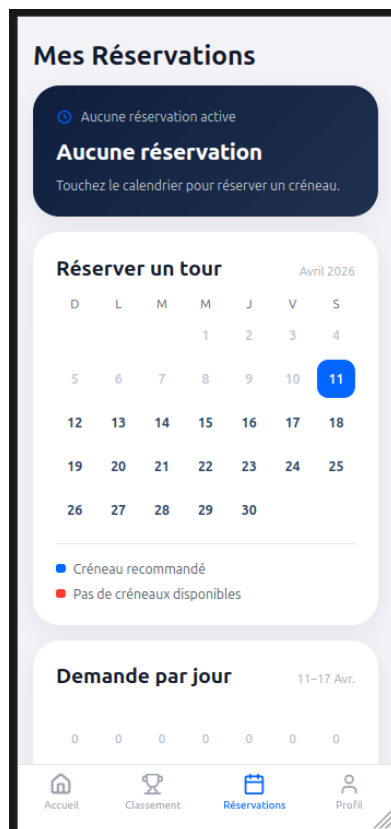
Cette interface a été pensée comme une expérience mobile complète : l'utilisateur peut y voir son prochain tour, consulter un aperçu du calendrier du mois, identifier les jours de plus forte ou de plus faible demande, puis ouvrir une fenêtre modale pour choisir précisément une date et une heure. L'ensemble garde ainsi une logique simple : *observer, décider, réserver*.

## Vue d'ensemble de l'écran

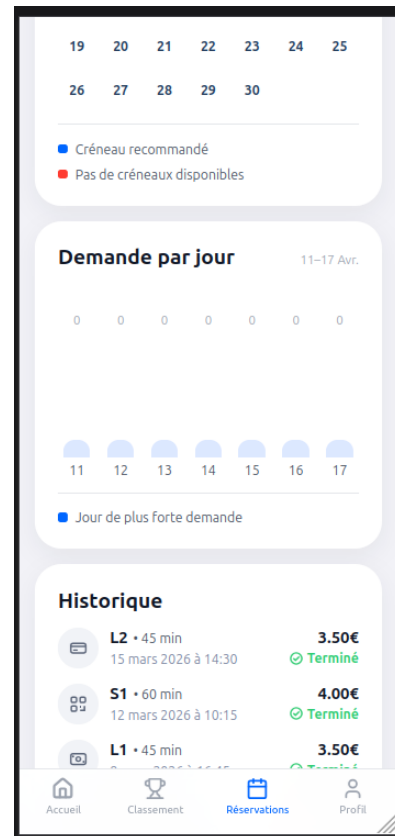
L'écran est organisé en quatre cartes principales superposées verticalement :

- une **carte de synthèse** affichant la prochaine réservation de l'utilisateur sous forme de compte à rebours ;
- une **carte calendrier** offrant une vue rapide du mois courant avec une légende visuelle ;
- une **carte statistique** représentant la demande sur les sept prochains jours ;
- une **carte d'historique** listant les dernières sessions avec leur durée, leur prix, leur statut et leur mode de paiement.

Cette hiérarchie visuelle permet à l'utilisateur de comprendre immédiatement sa situation actuelle, puis d'agir sans quitter la même vue.



(a) Partie supérieure de l'écran



(b) Partie inférieure de l'écran

FIGURE 12 – Vue d'ensemble de l'onglet Réservations avec la synthèse, le calendrier, la demande journalière et l'historique.

## La synthèse de la prochaine réservation

La première carte agit comme un rappel immédiat. Si une réservation est déjà active, l'utilisateur voit apparaître un **compte à rebours en heures et minutes** avant son prochain créneau, accompagné d'un résumé textuel comprenant le jour, l'heure et la machine concernée. Cette présentation répond à un besoin pratique : éviter d'oublier un créneau déjà réservé et réduire le stress lié à l'organisation du lavage.

Lorsque l'utilisateur n'a aucune réservation active, cette même carte change complètement d'état. Le compte à rebours disparaît et laisse place à un message indiquant qu'aucun créneau n'est actuellement réservé. Cette variation d'état rend l'interface plus explicite et évite les ambiguïtés.

## Le calendrier d'aperçu et la logique de recommandation

La seconde carte présente un **aperçu mensuel du calendrier**. Chaque jour est affiché sous forme de bouton compact. Une légende simple permet de comprendre la logique visuelle :

- **Bleu** : créneau recommandé ;
- **Rouge** : aucun créneau disponible ;
- **Gris clair** : jour passé ou non sélectionnable.

Ce choix graphique répond à un objectif ergonomique important : permettre à l'utilisateur de **repérer d'un coup d'œil les meilleures opportunités de réservation** sans devoir ouvrir immédiatement le sélecteur détaillé. La recommandation n'est pas arbitraire : elle repose sur une estimation de la demande, et le système met en avant le jour futur le moins chargé parmi ceux qui restent disponibles.

## La demande par jour

Sous le calendrier, une carte supplémentaire affiche la **demande journalière sur sept jours**. La représentation choisie est un histogramme vertical très simple, adapté à un écran mobile. Chaque colonne correspond à un jour et la barre la plus haute est mise en valeur en bleu afin d'indiquer le pic de demande.

L'objectif n'est pas de fournir une analyse statistique complexe, mais d'aider l'utilisateur à comparer rapidement les jours et à comprendre quels moments sont les plus sollicités. Cette information vient compléter la recommandation du calendrier : au lieu de réserver "à l'aveugle", l'étudiant peut visualiser la charge globale de la semaine.

## L'historique des réservations

La dernière carte contient l'**historique des 4 dernier réservations passées**. Chaque ligne affiche la machine utilisée, la durée de la session, la date, le prix payé ainsi que le statut final de la réservation. Une icône différente est affichée selon le mode de paiement utilisé, ce qui rend la lecture plus visuelle et facilite la distinction entre carte bancaire, espèces ou QR code.

Cette partie donne de la cohérence à l'expérience utilisateur car elle relie l'action présente de réservation à un historique concret d'utilisation. Elle joue aussi un rôle rassurant : l'étudiant peut vérifier qu'une réservation a bien été effectuée, terminée ou annulée.

## Réserver un nouveau créneau

Lorsque l'utilisateur touche la carte du calendrier, l'application ouvre une **fenêtre modale de réservation**. Cette fenêtre est structurée en deux étapes successives :

1. choisir une date ;

2. choisir une heure.

La partie supérieure de la modale reprend un calendrier mensuel plus détaillé. L'utilisateur peut naviguer d'un mois à l'autre grâce à des flèches latérales, mais aussi par un geste de glissement horizontal, ce qui renforce le caractère mobile de l'interface. Les jours passés sont automatiquement désactivés afin d'éviter toute réservation incohérente.

Une fois la date choisie, la partie inférieure affiche les **créneaux horaires disponibles**. Chaque horaire apparaît sous forme de bouton. Les créneaux déjà pris sont affichés dans un état bloqué, les créneaux passés sont désactivés, et le créneau sélectionné est mis en évidence en bleu. Le bouton de confirmation n'est activé que lorsque la sélection est complète, ce qui évite les erreurs de validation.

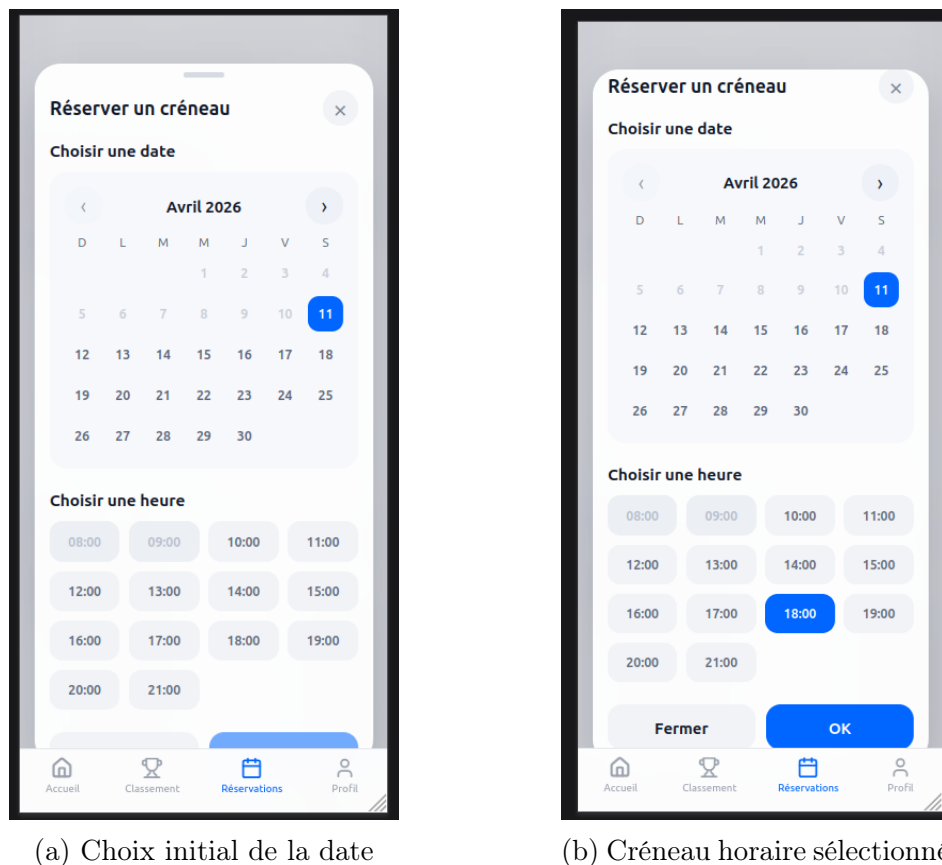


FIGURE 13 – Fenêtre modale de réservation permettant de choisir un jour puis une heure.

## Consulter et annuler sa réservation

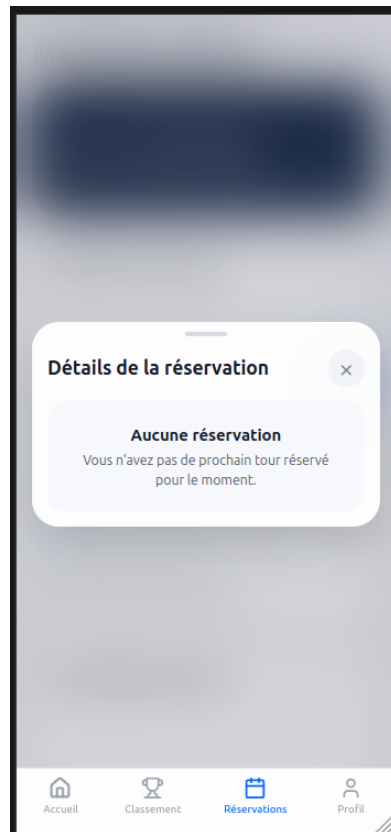
La carte de synthèse située en haut de l'écran est également interactive. En la touchant, l'utilisateur ouvre une seconde **fenêtre modale de détail**. Cette modale a deux états possibles :

- un état **informatif** lorsqu'une réservation est active ;
- un état **vide** lorsqu'aucune réservation n'est programmée.

Dans le premier cas, l'application affiche le nom de la machine, la date, l'heure, la durée et le prix. Un bouton rouge permet alors d'**annuler la réservation**. Dans le second cas, la modale joue surtout un rôle de confirmation visuelle : elle informe clairement l'utilisateur qu'aucun prochain tour n'est réservé. Cette distinction entre état plein et état vide améliore la lisibilité générale du système.



(a) Réservation active



(b) Aucune réservation en cours

FIGURE 14 – Fenêtre modale de détail de réservation selon les deux états possibles de l’application.

## Détails Techniques : Le code `reservations.js`

La logique interactive de cette page est répartie en deux fichiers complémentaires :

- `reservations.js`, chargé du rendu de l’interface et des interactions utilisateur ;
- `reservations.service.js`, chargé de gérer les données, la persistance locale et les opérations de réservation/annulation.

Cette séparation permet de conserver une architecture claire : la vue s’occupe d’afficher et d’écouter, tandis que le service s’occupe de stocker, vérifier et mettre à jour les données.

### Une vue pilotée par un état centralisé

Le fichier `reservations.js` repose sur un objet `state` contenant toutes les informations nécessaires au fonctionnement de l’écran : le mois affiché, la date sélectionnée, l’heure sélectionnée, les créneaux disponibles et la modale active.

Cette approche présente un avantage important : au lieu de recalculer manuellement chaque composant indépendamment, le script met à jour cet état puis relance les fonctions de rendu appropriées. L’écran reste donc cohérent même lorsqu’on change de mois, qu’on ouvre une modale ou qu’on réserve un créneau.

### Recommandation automatique d’une date

L’une des fonctions les plus intéressantes de cette vue est le calcul automatique du jour recommandé. Le système parcourt les jours du mois courant, ignore les dates passées ainsi que les journées saturées, puis met en avant le jour présentant la plus faible demande.

Listing 3 – Extrait simplifié de la logique de recommandation

```

1 function getRecommendedDateKey(monthDate) {
2     const todayKey = window.ReservationsService.todayKey();
3     let bestDateKey = null;
4     let bestDemandValue = Number.POSITIVE_INFINITY;
5
6     for (let day = 1; day <= lastDay; day += 1) {
7         const dateKey = toDateKey(new Date(
8             monthDate.getFullYear(),
9             monthDate.getMonth(),
10            day
11        ));
12
13        if (compareDateKeys(dateKey, todayKey) < 0) continue;
14
15        const demandValue = getDemandValue(dateKey);
16        if (demandValue >= 10) continue;
17
18        if (demandValue < bestDemandValue) {
19            bestDemandValue = demandValue;
20            bestDateKey = dateKey;
21        }
22    }
23    return bestDateKey;
24 }

```

Grâce à cette logique, l'application n'affiche pas seulement un calendrier statique : elle propose une aide à la décision directement intégrée à l'interface.

## Un système de réservation cohérent et persistant

Le service `reservations.service.js` joue le rôle d'une petite base de données locale. Il charge d'abord les données depuis le fichier `reservations.json`, vérifie leur validité, puis les conserve dans le navigateur via `localStorage`. Ainsi, lorsqu'un utilisateur change d'onglet ou recharge la vue, l'état de sa réservation reste cohérent.

Lorsqu'un créneau est confirmé, le service effectue plusieurs opérations successives :

- vérifier que la date et l'heure sont valides ;
- refuser les créneaux déjà passés ou déjà réservés ;
- retirer l'ancienne réservation éventuelle ;
- enregistrer le nouveau créneau dans la liste des slots occupés ;
- mettre à jour la demande journalière ;
- sauvegarder l'ensemble en mémoire locale.



Listing 4 – Principe simplifié de validation et de sauvegarde d’une réservation

```
1 async function reserveSlot(dateKey, time) {
2   if (!isFutureSlot(dateKey, time)) {
3     return { ok: false, message: "Impossible de réserver un
4       creneau passe." };
5   }
6   const daySlots = dbCache.bookedSlots[dateKey] || [];
7   if (daySlots.includes(time)) {
8     return { ok: false, message: "Ce creneau est deja pris." };
9   }
10
11   if (dbCache.nextReservation) {
12     updateDemand(dbCache.nextReservation.date, -1);
13   }
14
15   dbCache.nextReservation = { date: dateKey, time, ... };
16   updateDemand(dateKey, 1);
17   persist();
18
19   return { ok: true };
20 }
```

Cette logique garantit qu’une seule réservation active est associée à l’utilisateur à un instant donné, tout en gardant les statistiques de demande alignées avec l’état réel de la vue.

## Une interaction pensée pour le mobile

Au-delà du simple affichage, la vue a été conçue avec plusieurs détails d’interaction :

- ouverture et fermeture de modales sans rechargement de page ;
- fermeture au clic sur l’arrière-plan ;
- navigation clavier avec `Escape`, `ArrowLeft` et `ArrowRight` ;
- changement de mois par glissement horizontal ;
- adaptation responsive pour petits écrans.

Le fichier `reservations.css` complète cette logique avec une présentation très proche d’une application mobile native : fond flou derrière les modales, cartes arrondies, boutons larges et zones tactiles confortables. L’ensemble donne une sensation de fluidité cohérente avec le reste de l’application.

## Intégration avec le reste de l’application

Enfin, cette page s’intègre naturellement dans l’architecture SPA globale du projet. Elle peut être ouverte depuis la barre de navigation inférieure, mais aussi depuis le bouton principal de la page d’accueil. Dans ce deuxième cas, un indicateur global permet d’ouvrir automatiquement la modale de réservation dès l’arrivée sur la vue.

Ce détail montre que la page Réservations n’est pas un module isolé : elle est pensée comme le prolongement direct de l’écran d’accueil et comme l’un des centres fonctionnels majeurs de l’application.

# Système de Routage et Navigation

Pour que l'application soit rapide et fluide, un système de "routage dynamique" a été mis en place dans le fichier `app.js`. Contrairement à un site web classique, l'application ne recharge jamais la page entière.

## Changement de vue sans rechargement

Lorsqu'un utilisateur clique sur un bouton du menu, le JavaScript va chercher le fichier HTML correspondant et l'injecte directement dans la page. Un système de "cache" permet de mémoriser les pages déjà visitées pour qu'elles s'affichent instantanément la deuxième fois.

## Barre de navigation (Bottom Nav)

La barre située en bas de l'écran permet de passer d'une section à l'autre. Elle est gérée de façon automatique : le script détecte l'onglet actif et change sa couleur pour guider l'utilisateur.

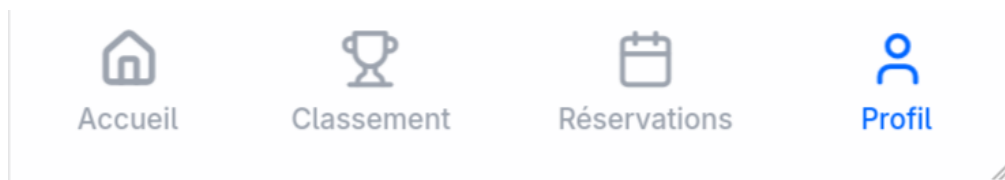


FIGURE 15 – Aperçu de la barre de navigation dynamique.

## La Page Profil

Le module de profil a été conçu pour ressembler à une application mobile native, tant par son fonctionnement que par son esthétique épurée.

## Design et Interface Visuelle

L'interface du profil se divise en trois zones distinctes pour une lecture rapide :

- **L'En-tête** : Affiche clairement le titre "Profil" accompagné d'une icône de notification (cloche) pour les alertes urgentes.
- **La Zone Utilisateur** : Met en avant l'identité de l'étudiant avec un avatar circulaire bordé aux couleurs de l'application. Les informations de contact (nom, e-mail, téléphone) sont présentées dans des blocs blancs arrondis, créant un contraste doux avec le fond légèrement grisé.
- **La Liste de Navigation** : Les différentes options de gestion sont organisées sous forme de liste avec des séparateurs fins et des icônes de flèches (chevrons), invitant l'utilisateur à explorer les sous-menus.

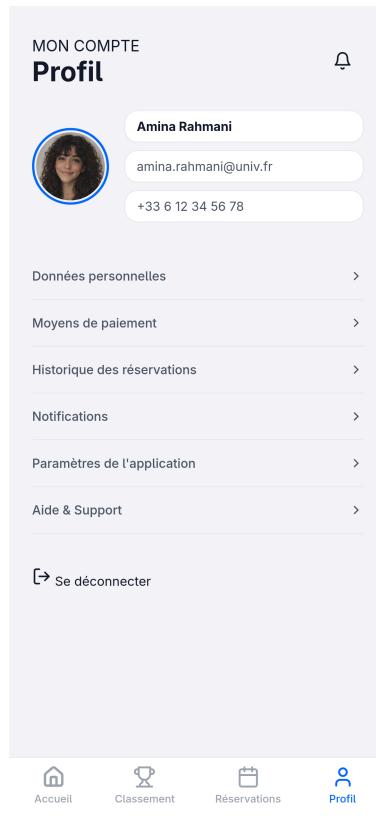


FIGURE 16 – Interface principale du compte utilisateur.

## Fonctionnement technique des sous-menus

Chaque section du profil répond à un besoin spécifique de l'étudiant, avec une gestion dynamique des données.

### Données personnelles et Moyens de paiement

Le sous-menu **Données** permet de consulter ses informations officielles (adresse, date de naissance). Pour le **Paiement**, un algorithme de sécurité masque les numéros de carte bancaire, ne laissant visibles que les chiffres nécessaires à l'identification de la carte (ex : 4502 ●●●● ●●●● 1234).

### Historique et Notifications

L'**Historique** permet de suivre les 24 dernières utilisations des machines avec le prix et le statut de chaque session. Les **Notifications** centralisent les messages système (fin de cycle, alertes de maintenance) avec un code couleur par type d'alerte.

### Paramètres et Support

La section **Paramètres** offre des options de personnalisation comme le mode sombre ou les préférences de notification via des commutateurs (toggles) interactifs. Enfin, **Aide & Support** propose une assistance directe et une foire aux questions pour résoudre les problèmes courants en laverie.

# Gestion et Origine des Données

- Le projet repose sur l'utilisation de fichiers JSON pour simuler une base de données réelle.
- **Création de fichiers** : Les structures des fichiers `profile.json` et `notifications.json` ont été créées de toutes pièces pour répondre aux besoins de l'interface de profil.
  - **Intégration technique** : Les fichiers `reservations.json` et `ranking.json` étaient déjà présents. Le travail a consisté à les connecter à l'interface en utilisant des fonctions asynchrones (`fetch` et `Promise.all`), ce qui permet de charger toutes les données en même temps avant d'afficher l'écran à l'utilisateur.



FIGURE 17 – Grille des six sous-menus intégrés dynamiquement dans le profil (Partie 1/2).

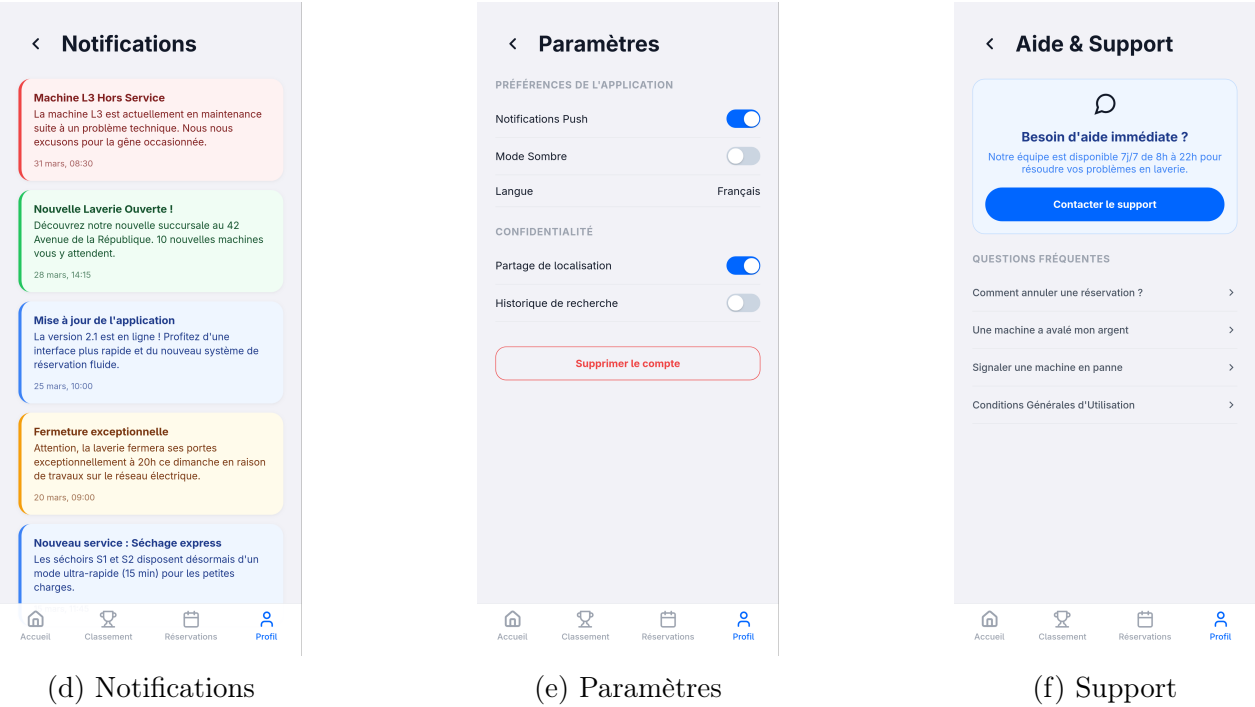


FIGURE 17 – Grille des six sous-menus intégrés dynamiquement dans le profil (Suite).